Unit 4: Database Integration
8 hours

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Contents

- Introduction to Databases
- Overview of relational (SQL) vs. NoSQL databases
- Database Connectivity with Flask
  - Connecting Flask applications to databases using SQLAlchemy
- Performing CRUD operations using HTML template
- Handling database connections and transactions

Introduction to Databases

- Databases allow you to store, retrieve, update, and manage data which is crucial for most web applications.
- In Flask, databases are used to store and manage data, making web applications more robust and versatile.
- Flask, a lightweight Python framework, can connect to various database systems like relational databases (SQL) and NoSQL databases, enabling you to persist data and manage it effectively.

Relational (SQL) vs. NoSQL databases

- Databases are essential for storing and managing application data.
- They come in two main types:
  - Relational (SQL) and
  - Non-relational (NoSQL)
- Each is designed to handle data in different ways.

Relational Database (SQL)

- **Relational Databases (SQL) are:**
  - MySQL,
  - PostgreSQL,
  - SQLite,
  - Oracle etc.
- **Key Features**:
  - Structured data stored in tables (rows and columns)
  - Fixed schema: Tables have predefined structure
  - Uses SQL (Structured Query Language) for querying
  - Relationships between tables using foreign keys

Relational Database (SQL)

Advantages

- Great for structured, consistent data
- Supports complex queries and joins
- Strong data integrity and ACID compliance

Use Cases:

- Banking systems
- Student information systems
- Inventory and e-commerce apps
- Any app requiring structured data and relations

Relational Database (SQL)

For example,

```
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    roll_number TEXT
);
```

```
CREATE TABLE attendance (
    id INTEGER PRIMARY KEY,
    student_id INTEGER,
    date DATE,
    FOREIGN KEY (student_id)
REFERENCES students(id)
);
```

NoSQL Databases

- A NoSQL database is a non-relational database designed to handle large volumes of unstructured, semi-structured, or structured data, offering flexibility, scalability, and performance for modern applications.
- Unlike traditional SQL databases, which use rigid table-based structures with predefined schemas, NoSQL databases support diverse data types and dynamic schemas, making them ideal for big data, real-time analytics, and distributed systems.
- Examples: MongoDB, CouchDB, Firebase, Cassandra

NoSQL Databases

Key Features:

- Flexible schemas good for unstructured or semi-structured data
- Stores data in various formats:
  - Document-based (MongoDB uses JSON-like documents)
  - Key-value pairs
  - Wide-column stores
  - Graph databases
- No need for predefined schema
- Scales easily across many servers (horizontal scaling)

NoSQL Databases

Advantages

- Ideal for dynamic or rapidly changing data
- Faster development without schema constraints
- Good for scalability and performance on large, distributed data

Use Cases

- Real-time analytics
- Content management systems
- IoT, mobile, and gaming applications
- Big data or unstructured data

NoSQL Databases

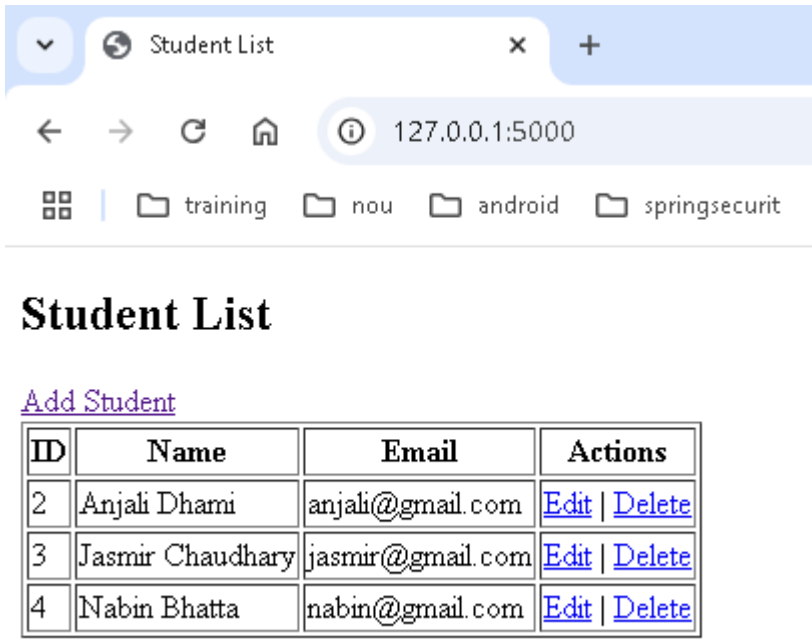
For example,

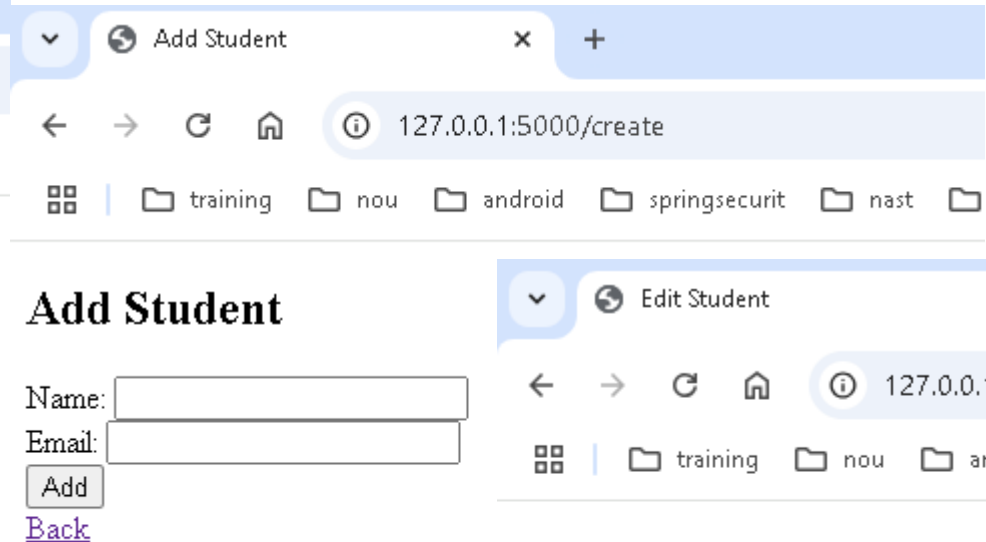
```
[{
  "name": “Jasmir",
  "roll": "BCA01",
  "attendance": [
    {"date": "2025-05-28", "status": "present"},
    {"date": "2025-05-27", "status": "absent"},
    {"date": "2025-05-26", "status": "present"}
  ]
},….
]
```

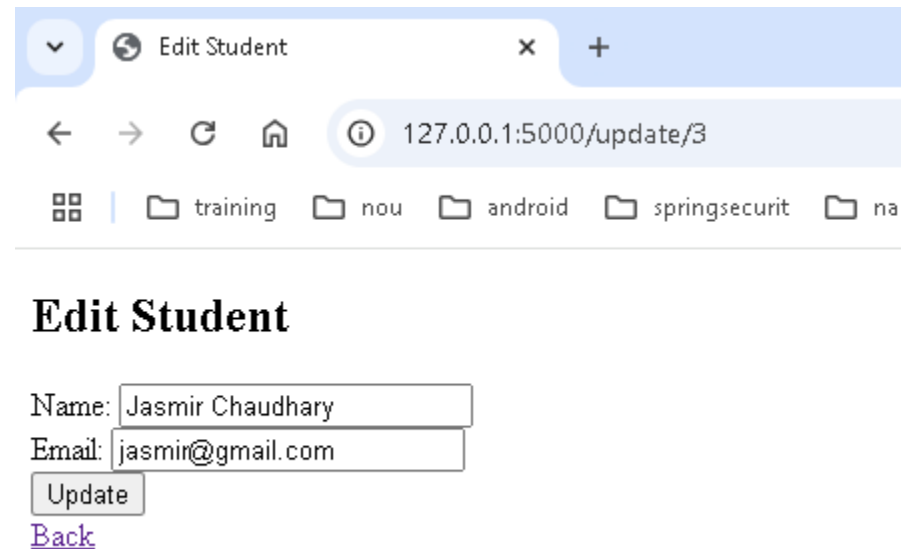
Database Connectivity with Flask

- Connecting Flask applications to databases using SQLAlchemy
- pip install flask
- pip install flask-mysqldb
- pip install flask-sqlalchemy pymysql

Performing CRUD operations using HTML template







Handling database connections and transactions

- Handling database connections and transactions properly is essential in Flask + SQLAlchemy apps to ensure data integrity, performance, and resource management.

- Here’s a breakdown of how to handle connections and transactions in various situations.

Handling database connections and transactions

Standard Way with Flask-SQLAlchemy

- When using Flask-SQLAlchemy, most connection and transaction handling is automatically managed for you via db.session.
- Basic Example

```
# Create a new record
new_user = User(name='Sunil')
db.session.add(new_user)
db.session.commit()
```

```
#update an existing record
user = User.query.filter_by(id=1).first()
user.name = 'Sunil Bahadur'
db.session.commit()
```

Handling database connections and transactions

Standard Way with Flask-SQLAlchemy

```
# Delete a record
user = User.query.get(1)
db.session.delete(user)
db.session.commit()
```

```
#rollback on error
try:
    user = User(name='Sunil')
    db.session.add(user)
    db.session.commit()
except Exception as e:
    db.session.rollback()
    print("Error:", e)
finally:
    db.session.close()  # Optional, closes the session
```

Handling database connections and transactions

Using session.begin() for Explicit Transactions

- You can use session.begin() for grouped transactional logic.

```
with db.session.begin():
user1 = User(name='User1')
user2 = User(name='User2')
db.session.add_all([user1, user2])
```

If any statement inside the block fails, the entire transaction is rolled back automatically.

Handling database connections and transactions

Manual Session and Connection Handling (Advanced)

- You can create a scoped session for more control:

```
from sqlalchemy.orm import scoped_session, sessionmaker
from sqlalchemy import create_engine

engine = create_engine('sqlite:///mydb.db')
Session = scoped_session(sessionmaker(bind=engine))

session = Session()
try:
    user = User(name='Test')
    session.add(user)
    session.commit()
except:
    session.rollback()
finally:
    session.close()

#use this when not using Flask-SQLAlchemy, or when needing manual transaction control.
```

Handling database connections and transactions

Contextual Session Management in Flask

- Flask-SQLAlchemy automatically uses a context-bound session, so you usually don’t need to manage session.close() unless you're working with threads or background tasks.
- To clean up explicitly, use the teardown:

```
@app.teardown_appcontext
def shutdown_session(exception=None):
    db.session.remove()
```

Next Chapter